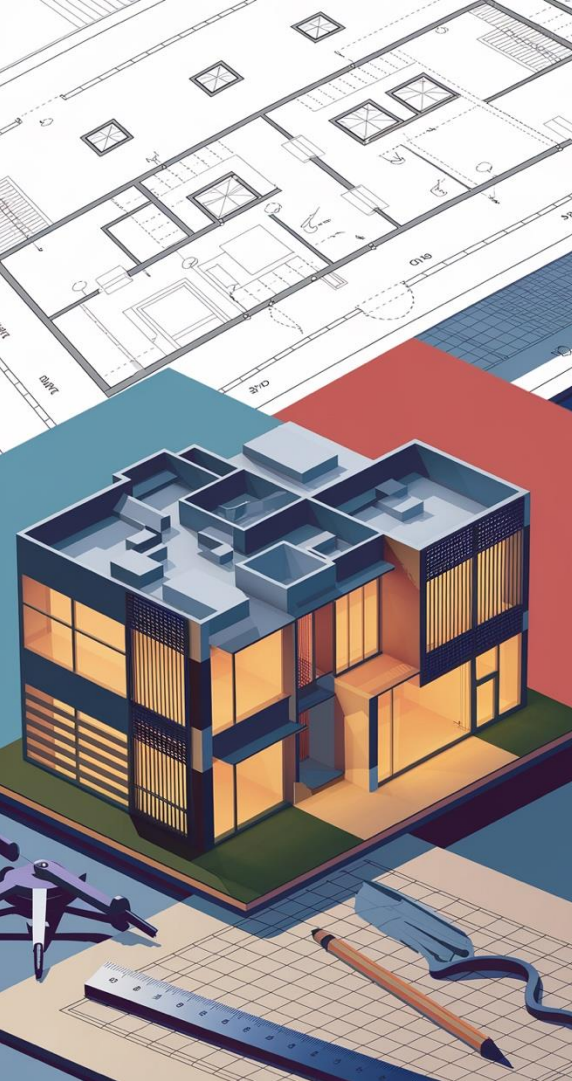


# Documentación de la Arquitectura

**Semana 08**

**Ingeniería de Software 2**

**2026-1**





## Logro de la sesión



El estudiantes son capaces de crear y estructurar documentación clara y coherente que represente adecuadamente la arquitectura de un sistema. Utiliza diferentes modelos y vistas, como el modelo 4+1, para comunicar efectivamente las decisiones de diseño, los componentes del sistema y sus interacciones.

- ¿Cómo crees que una documentación clara y bien estructurada puede influir en el éxito de un proyecto de software y en la colaboración entre los miembros del equipo?
- ¿Qué desafíos has enfrentado en proyectos anteriores debido a la falta de documentación adecuada?
- ¿De qué manera piensas que la comprensión de diferentes vistas arquitectónicas puede ayudarte a tomar decisiones más informadas y efectivas en el diseño de sistemas complejos?



## Contenidos

- Documentación de la arquitectura
- Modelo 4+1 vistas.
- Modelo C4.

# Documentación de la arquitectura

También denominado Descripción de Arquitectura que corresponde a un trabajo para expresar una arquitectura



# Documentación de la arquitectura

En todo momento, la documentación habla por el arquitecto:

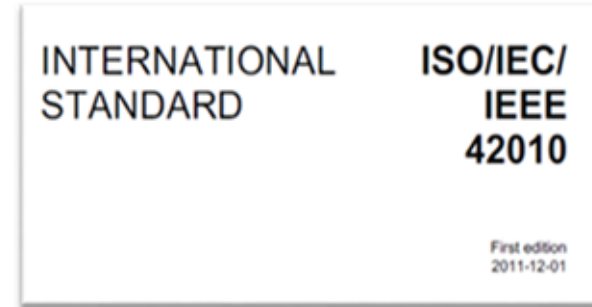
- **Hoy** - cuando el arquitecto puede hacer otras tareas relevantes en vez de responder preguntas sobre la arquitectura.
- **Mañana** - cuando el arquitecto haya dejado el proyecto y alguien más se encargue de su evolución y mantenimiento

Usualmente se hace porque alguien/algo la requiere - un contrato, un cliente, el proceso estándar de la empresa, etc.



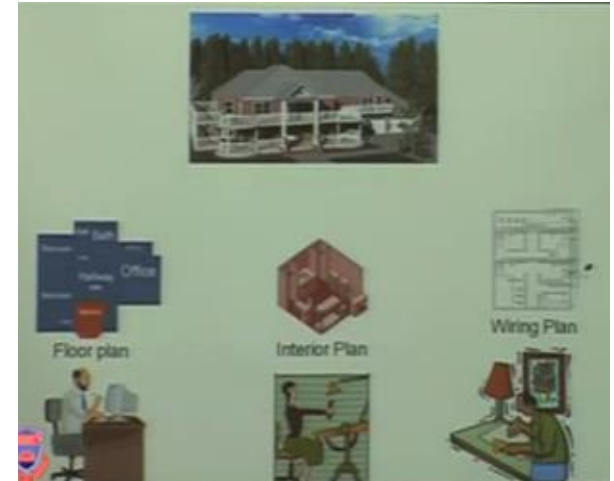
# Importancia de un documento de arquitectura

- Comunica aspectos esenciales del sistema.
- Es el punto de partida para planificar el proyecto.
- Es la base para analizar y evaluar alternativas de implementación de la arquitectura.
- Es esencial para realizar las actividades de mantenimiento y soporte técnico.



## Definiciones en la norma ISO/IEC 42010:2011

- La Descripción de una Arquitectura está conformada por una o varias **Vistas**.
- Una **Vista de Arquitectura** presenta las decisiones diseño (en forma de Modelos) relacionadas con un **Punto de Vista** sobre la Arquitectura.
- Un **Punto de Vista** concentra un grupo específico de preocupaciones de **Stakeholders** sobre el sistema.
  - *Preocupaciones:* rendimiento, seguridad, escalabilidad, usabilidad, facilidad de modificación, escalabilidad, entre otras.





## Vistas y puntos de vista

Arquitectura del software = entidad compleja

- No puede describirse con una dimensión simple
- Requiere varias vistas para diferentes *stakeholders*

Vista = Representación de un sistema respecto a algunas preocupaciones

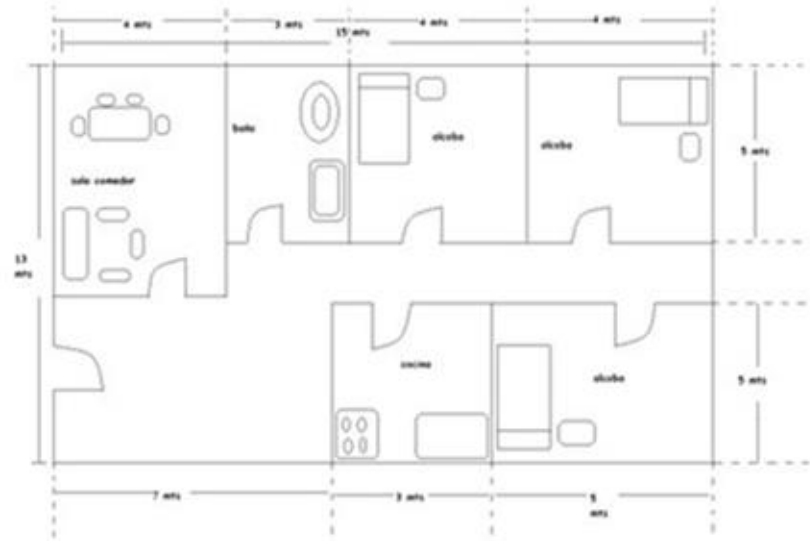
- Diferentes vistas sirven para diferentes objetivos

Punto de vista = Colección de patrones, plantillas y convenciones para construir una vista

- Ejemplos: estructura, comportamiento, despliegue

# Vistas Arquitectónicas

- Una vista es una perspectiva del sistema.
- Es un subconjunto de los elementos de la arquitectura.
- Describen un comportamiento particular del sistema.



# Vistas

El concepto de vista es el principio fundamental para organizar la documentación de arquitectura:

- además de las vistas, incluye información adicional que pertenece al documento de arquitectura
- no es un método
- no incluye una secuencia de pasos a seguir
- es una colección de técnicas que llevan una filosofía subyacente:

**La documentación de arquitectura debe ser útil a las personas que dependen de ella para realizar su trabajo**



# Preocupaciones

- Fuente: ISO/IEC 42010:2011

| Name                   | Description                                                                                                                                                                                                       | Use for Architecture Documentation                                                                                                                                                                  |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Architect              | Responsible for the development of the architecture and its documentation. Focus and responsibility is on the system.                                                                                             | Negotiating and making trade-offs among competing requirements and design approaches. A vessel for recording design decisions. Providing evidence that the architecture satisfies its requirements. |
| Business manager       | Responsible for the functioning of the business/organizational entity that owns the system. Includes managerial/executive responsibility, responsibility for defining business processes, and more.               | Understanding the ability of the architecture to meet business goals.                                                                                                                               |
| Conformance checker    | Responsible for assuring conformance to standards and processes to provide confidence in a product's suitability.                                                                                                 | Basis for conformance checking, for assurance that implementations have been faithful to the architectural prescriptions.                                                                           |
| Customer               | Pays for the system and ensures its delivery. The customer often speaks for or represents the end user, especially in a government acquisition context.                                                           | Assuring required functionality and quality will be delivered, gauging progress, estimating cost, and setting expectations for what will be delivered, when, and for how much.                      |
| Database administrator | Involved in many aspects of the data stores, including database design, data analysis, data modeling and optimization, installation of database software, and monitoring and administration of database security. | Understanding how data is created, used, and updated by other architectural elements, and what properties the data and database must have for the overall system to meet its quality goals.         |

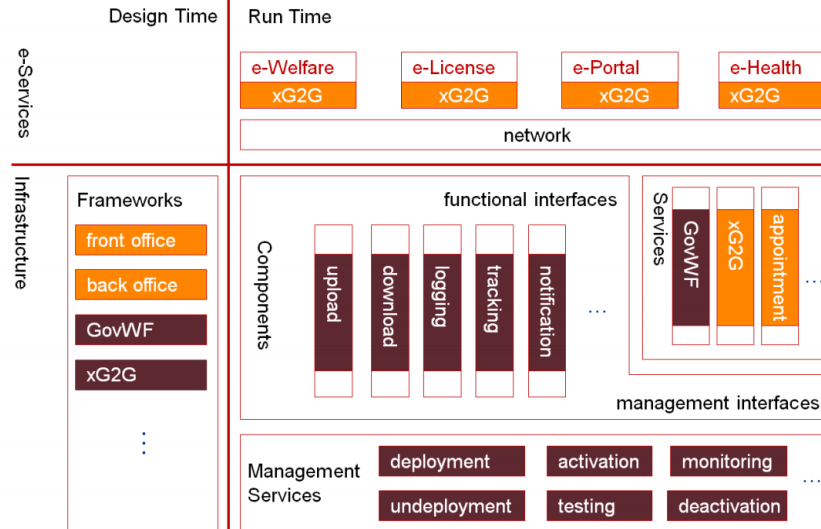
# Notaciones

- 1) Notaciones informales
- 2) Notaciones semi-formales
- 3) Notaciones formales

# Notaciones Informales

- vistas representadas usando diagramas de propósito general
- la semántica de la descripción está en lenguaje natural y no puede ser formalmente analizada

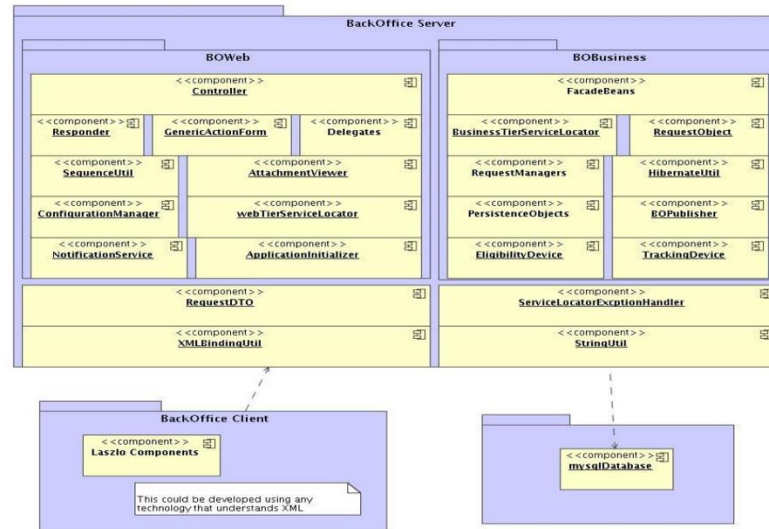
ejemplo: PowerPoint



# Notación Semi - Formal

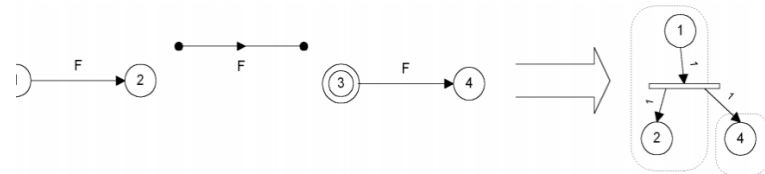
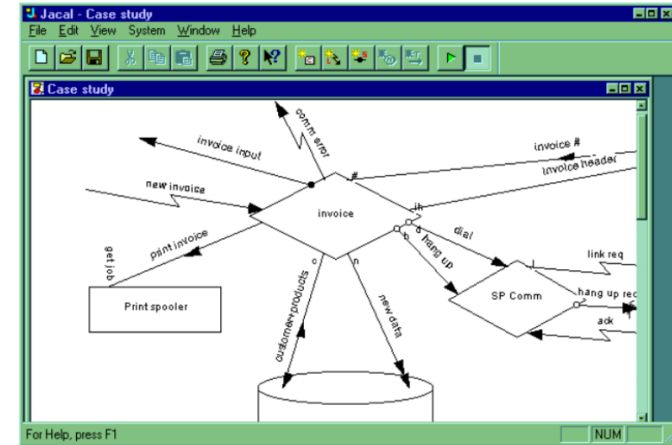
- las vistas se expresan en una notación estandarizada que utiliza elementos gráficos y reglas de construcción no provee un tratamiento semántico completo del significado de los elementos

- ejemplo: UML



# Notación Formal

- las vistas son descritas en una notación que tiene una semántica precisa (generalmente basada en matemáticas)
- es posible un análisis formal tanto sintáctico como semántico
- conocidas como Architecture Description Languages (ADLs), proveen un vocabulario gráfico y una semántica subyacente para representar arquitecturas
- soportan automatización a través de herramientas asociadas



Ref: <http://www-2.dc.uba.ar/profesores/nicok/jacal.htm>



# Modelo de Vista Arquitectónica 4+1



Fuente:

<https://www.cs.ubc.ca/~gregor/teaching/papers/4+1view-architecture.pdf>

## Múltiples vistas en una arquitectura

- **Vista Lógica:** Abstracciones Claves del Sistema. Relaciones requerimientos del sistema con entidades.
- **Vista de Proceso:** Procesos en interacción. Permite evaluar características no funcionales del sistema como rendimiento y disponibilidad.
- **Vista de Desarrollo:** Muestra la descomposición del software. A nivel de desarrollo, cada componente debería de ser implementado por un desarrollador o equipo de desarrollo.
- **Vista Física:** Expone el hardware del sistema. Muestra la relación entre componentes y hardware.

## Un sistema – varias vistas



# Arquitecturas de Transición

Estado actual y futuro de una arquitectura



# AS-IS vs. TO-BE

## Arquitectura de Transición

- **AS - IS:** Estado actual de la arquitectura en cada una de las vistas arquitectónicas.
- **TO - BE:** Estado futuro de la arquitectura en cada una de las vistas arquitectónicas
- **Transición:** Cuando la evolución del AS-IS al TO-BE es complejo

# AS-IS vs. TO-BE

## Arquitectura de Transición

- **AS-IS** : Se refiere al estado actual de un sistema o proceso. En este contexto, se analiza y documenta cómo funciona el sistema en su forma actual, incluyendo sus características, limitaciones y problemas. Este análisis es fundamental para identificar áreas de mejora y establecer una base para el desarrollo futuro.
- **TO-BE** : Describe el estado futuro deseado de un sistema o proceso. Implica la conceptualización de cómo debería funcionar el sistema una vez que se implementen las mejoras necesarias. Este estado ideal se alinea con los objetivos del negocio y las necesidades de los usuarios, sirviendo como guía para el diseño y desarrollo de soluciones.
- **Arquitectura de Transición** : La Arquitectura de Transición es el plan que detalla los pasos y estrategias necesarios para mover un sistema del estado AS-IS al estado TO-BE. Incluye la identificación de hitos, recursos requeridos, riesgos potenciales y métodos de implementación. Este enfoque asegura que la transición se realice de manera ordenada y efectiva, minimizando interrupciones y maximizando la adopción de las nuevas soluciones.

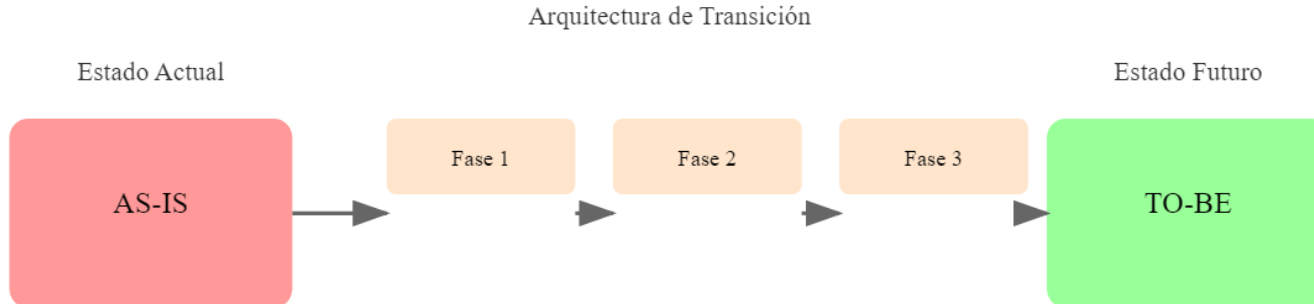
# Arquitecturas de Transición

- Conjunto de Arquitecturas intermedias entre el AS-IS y el TO-BE.
- Representa una arquitectura en un momento determinado.
- Utilizadas cuando el proceso de transición entre el AS-IS y el TO-BE es muy complejo
- Complejo a nivel:
  - Tecnológico
  - Gobierno
  - Tiempo
  - Recursos
- Arquitecturas de Transición
  - Implementación de Middleware.
  - Componentes redundantes (antiguo vs. nuevo).
  - Creación de nuevas interfaces.

# Grafica de una Arquitecturas de Transición

Para graficar una Arquitectura de Transición, se puede representar un diagrama que muestre el estado actual (AS-IS), el estado futuro (TO-BE) y los pasos intermedios necesarios para la transición. A continuación, describo cómo se podría estructurar el diagrama:

1. **Estado Actual (AS-IS):** Un bloque que representa el sistema actual, con sus componentes y limitaciones.
2. **Pasos de Transición:** Flechas o bloques intermedios que indican las etapas o hitos necesarios para realizar la transición. Estos pueden incluir actividades como análisis de requisitos, diseño de la nueva arquitectura, implementación de cambios, pruebas y capacitación.
3. **Estado Futuro (TO-BE):** Un bloque que representa el sistema deseado, mostrando cómo se verán los componentes y las mejoras implementadas.





# Propuesta de un documento de arquitectura

Resaltar aspectos importantes de un documento de arquitectura



# Documento de Arquitectura

## 1. Metas de la arquitectura

- [código] meta 1
- ....

## 2. Escenarios de atributos de calidad

- [código] escenario de atributo de calidad 1
- ...

## 3. Restricciones

- [código] restricción 1
- ....

## 4. Estilos arquitecturales

| Estilo   | Justificación                                         |
|----------|-------------------------------------------------------|
| <nombre> | <explicar las razones por las que se elige el estilo> |

# 1. Meta de la arquitectura

## Ejemplos de metas

- Los metas arquitecturales proporcionan la motivación y el fundamento de las decisiones de arquitectura.
- Definen cómo el sistema debe responder a los cambios a lo largo del tiempo.

More about of [The goals of Software Architecture](#)

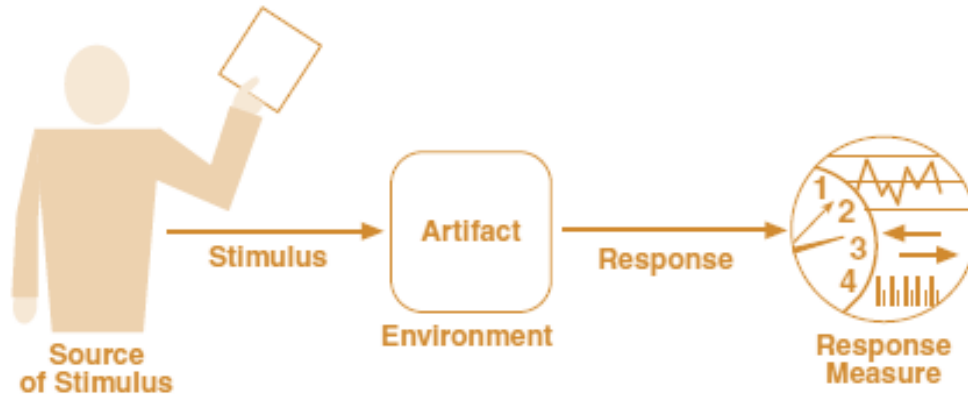
| Código                                    | Meta                                                                                                    |
|-------------------------------------------|---------------------------------------------------------------------------------------------------------|
| AG001 Interoperability                    | The WSA should enable the development of interoperable Web services across a wide array of environments |
| AG002 Reliability                         | The WSA must be reliable and stable over time                                                           |
| AG003 Integration with the World Wide Web | The WSA must be consistent with the current and future evolution of the World Wide Web.                 |

Source: [WSA: Web Service Architecture](#)



## **2. Escenarios de atributos de calidad**

## Escenarios de atributos de calidad



- Un atributo de calidad (QA) es una propiedad medible o comprobable de un sistema que se utiliza para indicar qué tan bien el sistema satisface las necesidades de los stakeholders.
- Atributos de calidad: usabilidad, confiabilidad, desempeño, etc.
- Tiene que ser descritos usando escenarios de atributos de calidad.

[Ver video](#)

## Ejemplo: Escenarios de atributos de calidad

| Escenario de calidad   | #1                                                                     |
|------------------------|------------------------------------------------------------------------|
| Atributo de calidad    | Desempeño                                                              |
| Fuente del estímulo    | Cuando 1000 profesores de forma concurrente                            |
| Estímulo               | Suben un archivo de 15 MB                                              |
| Entorno                | Con una velocidad de subida en internet entre 1 y 1.5 Mbps             |
| Artefacto              | A través de la página para subir archivos del sistema del aula virtual |
| Respuesta              | El archivo es subido y puesto a disposición de lo estudiantes          |
| Medida de la respuesta | 95 % de los archivos están disponibles en menos de 5 segundos.         |

### 3. Restricciones

Las restricciones son decisiones de diseño de la arquitectura que ya han sido tomadas.



# Restricciones

Ejemplos de restricciones:

- Lenguaje de programación.
- Base de datos.
- Infraestructura.
- Estilo arquitectónico (Por ejemplo: SOA).
- Otros.

1. *Functional requirements.* These requirements state what the system must do, and how it must behave or react to runtime stimuli.
2. *Quality attribute requirements.* These requirements are qualifications of the functional requirements or of the overall product. A qualification of a functional requirement is an item such as how fast the function must be performed, or how resilient it must be to erroneous input. A qualification of the overall product is an item such as the time to deploy the product or a limitation on operational costs.
3. *Constraints.* A constraint is a design decision with zero degrees of freedom. That is, it's a design decision that's already been made. Examples include the requirement to use a certain programming language or to reuse a certain existing module, or a management fiat to make your system service oriented. These choices are arguably in the purview of the architect, but external factors (such as not being able to train the staff in a new language, or having a business agreement with a software supplier, or pushing business goals of service interoperability) have led those in power to dictate these design outcomes.

## 4. Estilos arquitecturales

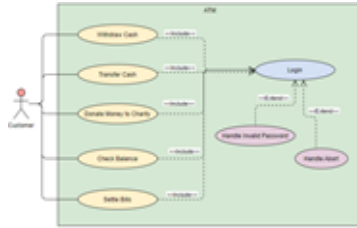
# Estilos arquitecturales

- Mencionar los estilos arquitecturales elegidos y la justificación.
- Estilos arquitecturales: SOA, N-capas, Microservicios, Otros

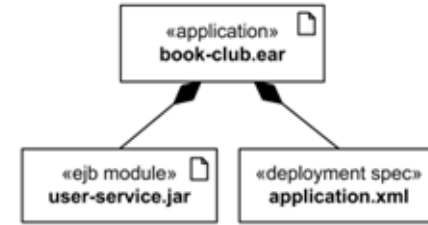
| Estilo                | Justificación                                         |
|-----------------------|-------------------------------------------------------|
| <nombre del estilo 1> | <explicar las razones por las que se elige el estilo> |
| <nombre del estilo 2> | <explicar las razones por las que se elige el estilo> |
|                       |                                                       |

# Vistas en el Documento de Arquitectura

# Documento de Arquitectura



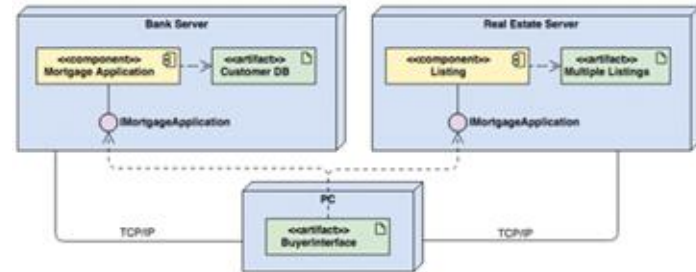
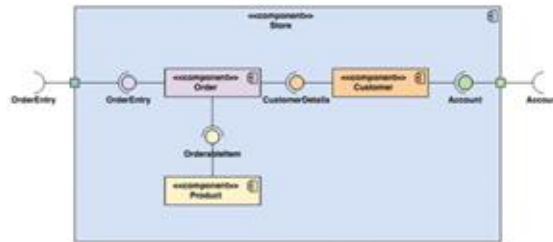
**Vista funcional**



**Vista de implementación**



**Vista lógica**



**Vista de despliegue**



# Vista funcional

Se utiliza para presentar los requisitos significativos para la arquitectura

## Documentación de la vista funcional

- Uno o varios diagramas de casos de uso mostrando los requisitos funcionales (casos de uso o **historias**) significativas para la arquitectura.
- Información relevante sobre los requisitos: código, propósito, actores, etc.

Nota. - Para el caso del curso utilizamos **Historias de usuario**

| Nro                       | Título       |
|---------------------------|--------------|
|                           | Descripción  |
| Prioridad                 | Dependencias |
| Criterios de Satisfacción |              |

| 1                                                                                                                                                                                                                                                                                         | Pagar un pedido |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------|
| Como un cliente, quiero poder pagar con una tarjeta de crédito para confirmar en forma inmediata la compra                                                                                                                                                                                |                 |
| Alta                                                                                                                                                                                                                                                                                      | 4,5             |
| <ul style="list-style-type: none"><li>- Aceptar tarjeta Visa, Mastercard y American Express</li><li>- Conectarse a la aplicación de seguridad de cada tarjeta que valide los datos ingresados</li><li>- Mostrar un mensaje de éxito o error como resultado del proceso de pago.</li></ul> |                 |

## Vista lógica

Se utiliza para mostrar lógicamente como está organizado y estructurado la arquitectura. También, para satisfacer los requerimientos funcionales y los atributos de calidad.



## Vista Lógica

- Permite **organizar el modelo de diseño en piezas más manejables.**  
Subsistemas e Interfaces.
- **No todos** los subsistemas se implementan.
- **Identificación de Subsistemas** de la aplicación:
  - **Se hacen a partir de los paquetes del análisis**, los que se refinan si es necesario.

## Consideraciones de diseño para *vista lógica*

Para el diseño de la vista lógica tener en cuenta:

- Las metas de la arquitectura.
- Los escenarios de atributos de calidad.
- Los requisitos funcionales significativos.
- Restricciones (Ejemplo Cloud con AWS).
- Estilos arquitectónicos seleccionados (capas, orientada a servicios, orientación a objetos, otros). **Los estilos arquitecturales guían el diseño.**

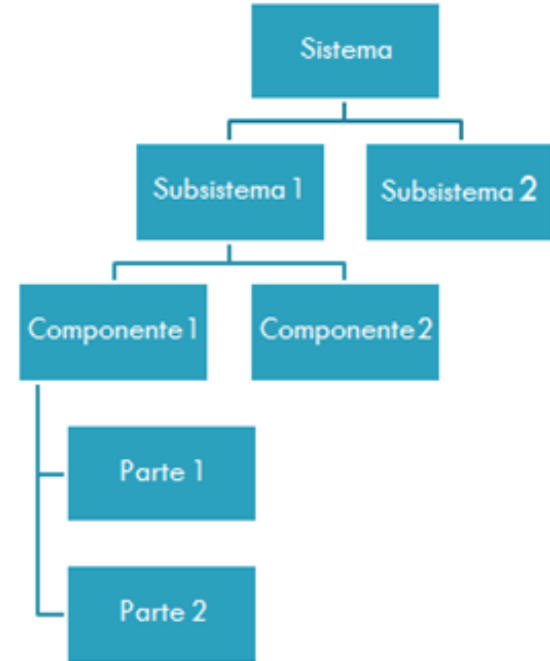
## Niveles de descomposición - *vista lógica*

Útil para determinar el nivel de detalle de la arquitectura.

- Sistema
- Subsistema
- Componentes
- Parte
  - Interfaz en Java
  - Clase en Java
  - Archivo XML
  - Archivo de propiedades

Esto es arquitectura

Esto no es arquitectura

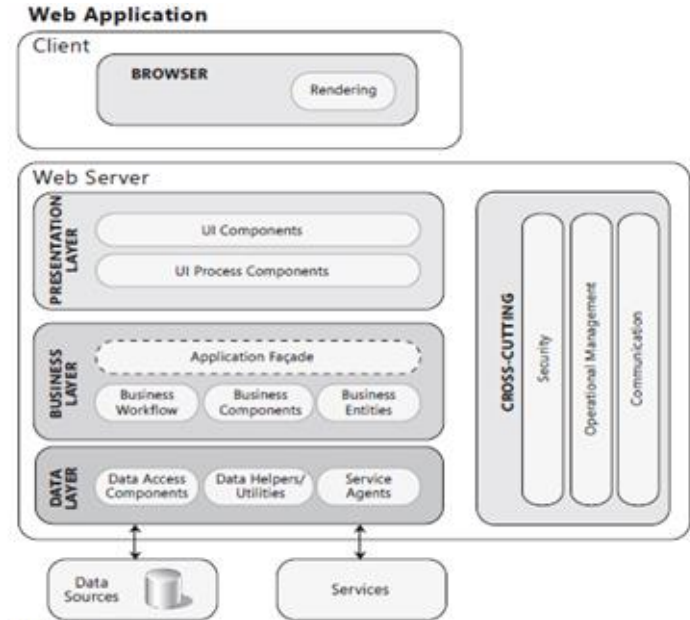
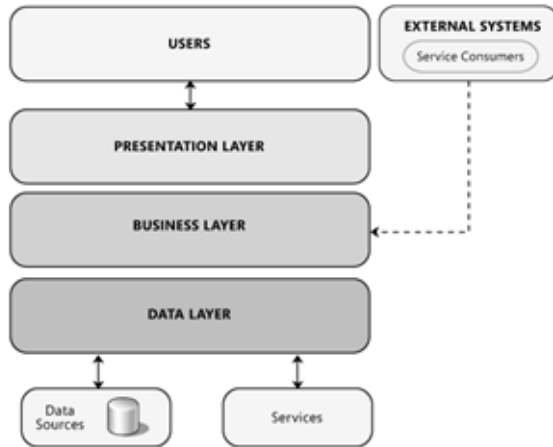


Fuente: [Nasa](#)



# Cómo aplicar un estilo de arquitectura a la vista lógica

Arquitectura N-capas aplicada a una aplicación web

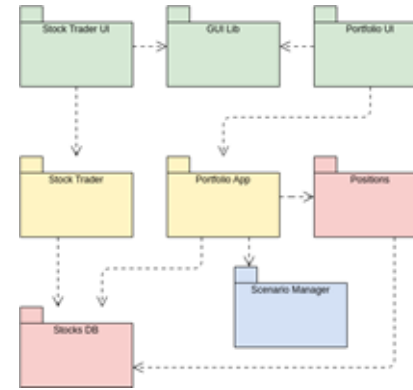
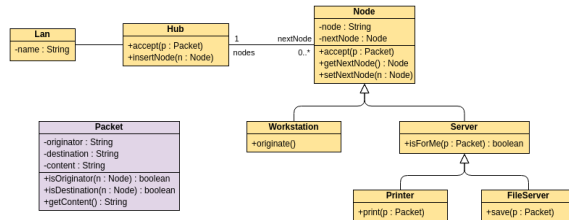


**Figure 1**  
The typical structure of a Web application

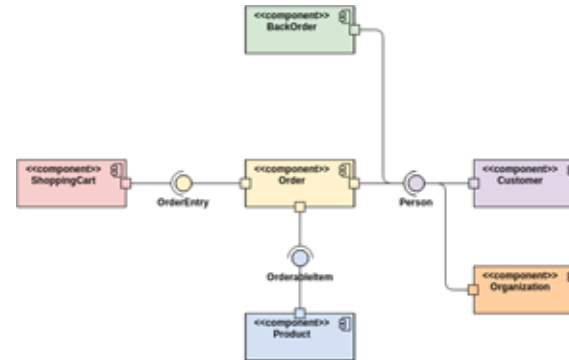
Fuente: [Microsoft Application Architecture Guide, 2nd Edition](#)

# ¿Cómo modelar la estructura de la solución?

- Diagrama de Clases
- Diagrama de paquetes de UML
- Diagrama de componentes de UML



<https://online.visual-paradigm.com/diagrams/templates/package-diagram/>



<https://online.visual-paradigm.com/diagrams/templates/component-diagram/>

## Contenido referencial de la vista lógica

| Tema                              | Nivel      | Contenido                                                                                                                                                                                                                  |
|-----------------------------------|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Visión general de la arquitectura | Sistema    | Diagramas de paquetes mostrando los subsistemas (según los estilos arquitecturales), principios de diseño, información general sobre cada subsistema, decisiones de arquitectura, restricciones, etc.                      |
| Arquitectura de cada subsistema   | Subsistema | Diagramas de paquetes mostrando la organización de cada subsistema (según los estilos de arquitectura), principios de diseño, información relevante sobre cada subsistema, decisiones de arquitectura, restricciones, etc. |
| Comportamiento                    | Clase      | Diagramas de clase contemplando los escenarios de atributos de calidad se deben tomar en cuenta, información relevante sobre cada componente, decisiones de arquitectura, restricciones, etc.                              |

**Granularidad**

Menor

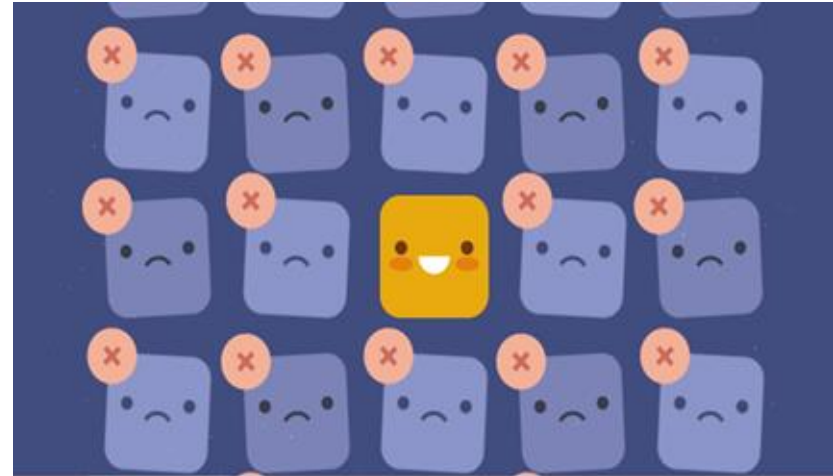


Mayor



## Errores más comunes en la vista lógica

- No conocer en profundidad los estilo arquitectónicos.
- Diagramas genéricos.
- Considerar sólo los requisitos de negocio.
- Uso incorrecto de la notación UML.
- No explicar los diagramas.



## Vista de implementación / componentes

La organización real de los módulos de software en el entorno de desarrollo.  
Muestra cómo se empaqueta el código fuente, las librerías, los ejecutables y sus dependencias (la arquitectura de software "física" del código).

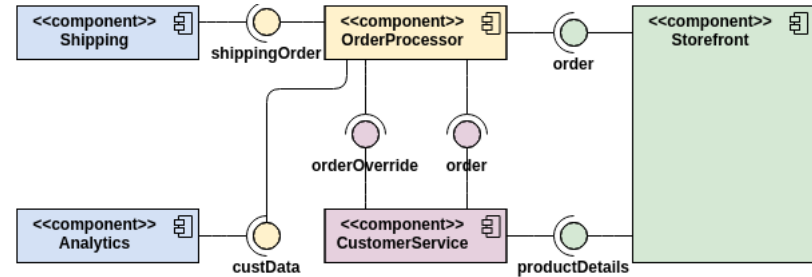


## Vista de Implementación

- **Modela los componentes y archivos** que utilizan para ensamblar y hacer disponible el sistema físico
- Contiene visión general del Modelo de Implementación y su organización en módulos en paquetes y capas .
- Emplea los diagramas :
  - Para modelar **aspectos estáticos** : **Diagrama de Componentes**.
  - Para modelar **aspectos dinámicos**:
    - Diagrama de Interacción
    - Diagrama de estados
    - Diagrama de actividades

# ¿Cómo modelar la implementación?

- Contenido:
  - Unidades de despliegue y sus interdependencias.
    - Diagrama de componente en UML 2.0
    - Diagrama de Paquetes: (Usado aquí para mostrar la jerarquía de directorios, librerías o subsistemas del código).



## Contenido referencial de la vista de implementación

| Tema                                                      | Contenido                                                                                                                                                      |
|-----------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Restricciones a la implementación</b>                  | Relación de restricciones consideradas en la vista de implementación                                                                                           |
| <b>Escenarios de atributos de calidad</b>                 | Relación de escenarios de atributos de calidad considerados en la vista de implementación                                                                      |
| <b>Unidades de implementación y sus Interdependencias</b> | Diagramas de Despliegue y componentes mostrando las unidades a desplegar(artefactos) y sus interdependencias, información relevante sobre cada artefacto, etc. |

## Vista de despliegue

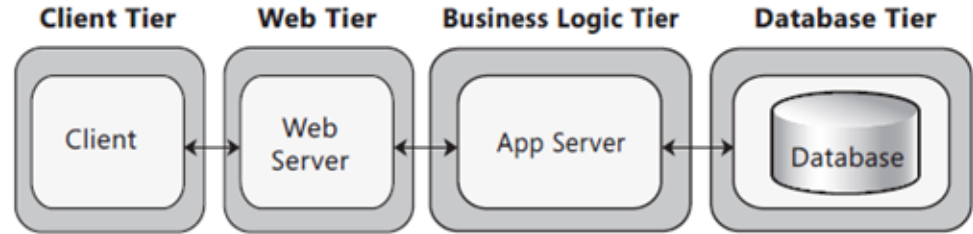
Presenta el entorno de ejecución necesario para ejecutar el software conforme a los requerimientos de disponibilidad, rendimiento, escalabilidad, entre otros; así como la distribución de las unidades de despliegue en el entorno de ejecución.

## Vista de Despliegue.

- **Modela los nodos de la topología de hardware** sobre la que se ejecutan los sistemas.
  - Para modelar **aspectos estáticos** : Diagrama de Despliegue
  - Para modelar **aspectos dinámicos**.
    - Diagramas de Interacción
    - Diagramas de Estados
    - Diagrama de Actividades

## Patrón N-Tier

- El patrón de N-Niveles representa un patrón general de distribución donde los componentes de la aplicación se separan a través de uno o más nodos físicos.



**Figure 6**

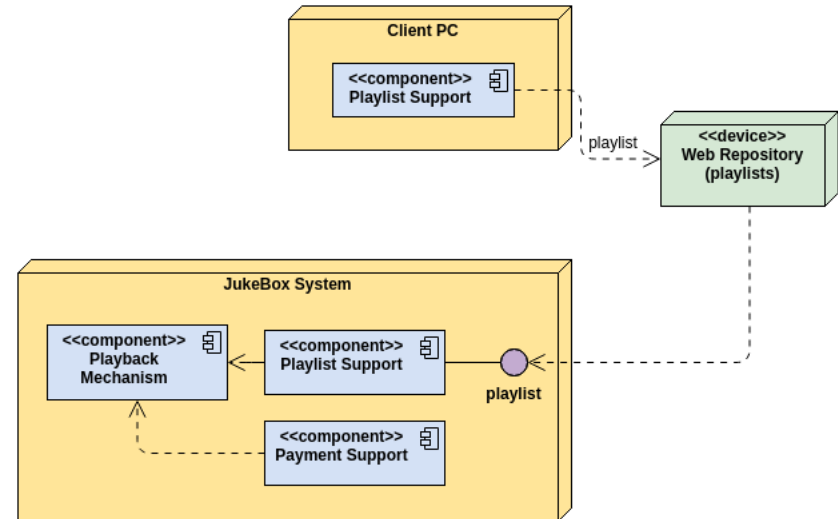
*4-tier deployment with the Web code and the business logic on separate tiers*

Consider the 4-tier pattern if security requirements dictate that business logic cannot be deployed to the perimeter network, or you have application code that makes heavy use of resources on the server and you want to offload that functionality to another server.

Fuente: [Microsoft Application Architecture Guide, 2nd Edition](#)

# ¿Cómo modelar el despliegue?

- Contenido
  - Infraestructura de software y hardware necesaria para la ejecución del sistema, así como la ubicación de las unidades de despliegue
- Representación gráfica
  - Diagrama de despliegue en UML 2.0



## Contenido referencial de la vista de despliegue

| Tema                                      | Contenido                                                                                                                                                                                                        |
|-------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Restricciones a la implementación</b>  | Relación de restricciones consideradas en la vista de despliegue                                                                                                                                                 |
| <b>Escenarios de atributos de calidad</b> | Relación de escenarios de atributos de calidad considerados en la vista de despliegue                                                                                                                            |
| <b>Principios de diseño</b>               | Principios de diseño empleado en la vista de despliegue                                                                                                                                                          |
| <b>Infraestructura de ejecución</b>       | Diagramas de despliegue mostrando la infraestructura de software y hardware necesaria para ejecutar la solución, información relevante sobre cada nodo, protocolos de comunicación, explicación de los diagramas |



## Vista de Proceso

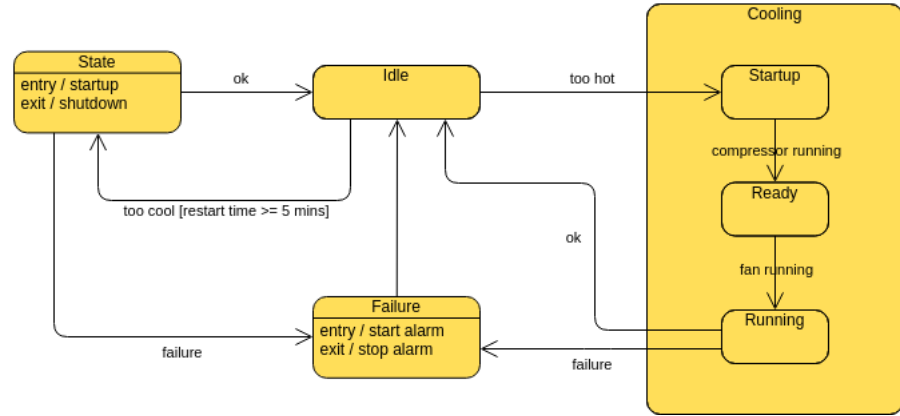
La arquitectura de procesos toma en cuenta algunos requisitos no funcionales tales como la performance y la disponibilidad. Se enfoca en asuntos de concurrencia y distribución, integridad del sistema, de tolerancia a fallas.

## Vista de Proceso

- En esta vista **se describe las tareas sus interacciones y la asignación de objetos del diseño a las tareas.**
- Muestra elementos **relacionados con el desempeño, escalabilidad, concurrencia**
- Se usa solo si el sistema tiene alta concurrencia, por lo que es una vista **OPCIONAL**
- Modela la Distribución de los procesos e hilos
- Emplea los mismos diagramas de las vistas de diseño, pero poniendo atención en las clases activas y los objetos que presentan hilos y procesos.

# ¿Cómo modelar?

- Representación gráfica
  - Diagrama de secuencia
  - Diagrama de actividades
  - Diagrama de estados



## Modelo C4

El modelo C4 es un enfoque de "primero la abstracción" para diagramar la arquitectura de software, basado en abstracciones que reflejan cómo los arquitectos y desarrolladores de software piensan y construyen software



## Modelo C4

El pequeño conjunto de abstracciones y tipos de diagramas hace que el modelo C4 sea fácil de aprender y utilizar. Tenga en cuenta que no es necesario utilizar los 4 niveles del diagrama; solo aquellos que agregan valor: los diagramas de contexto del sistema y de contenedor son suficientes para muchos equipos de desarrollo de software.

Describe:

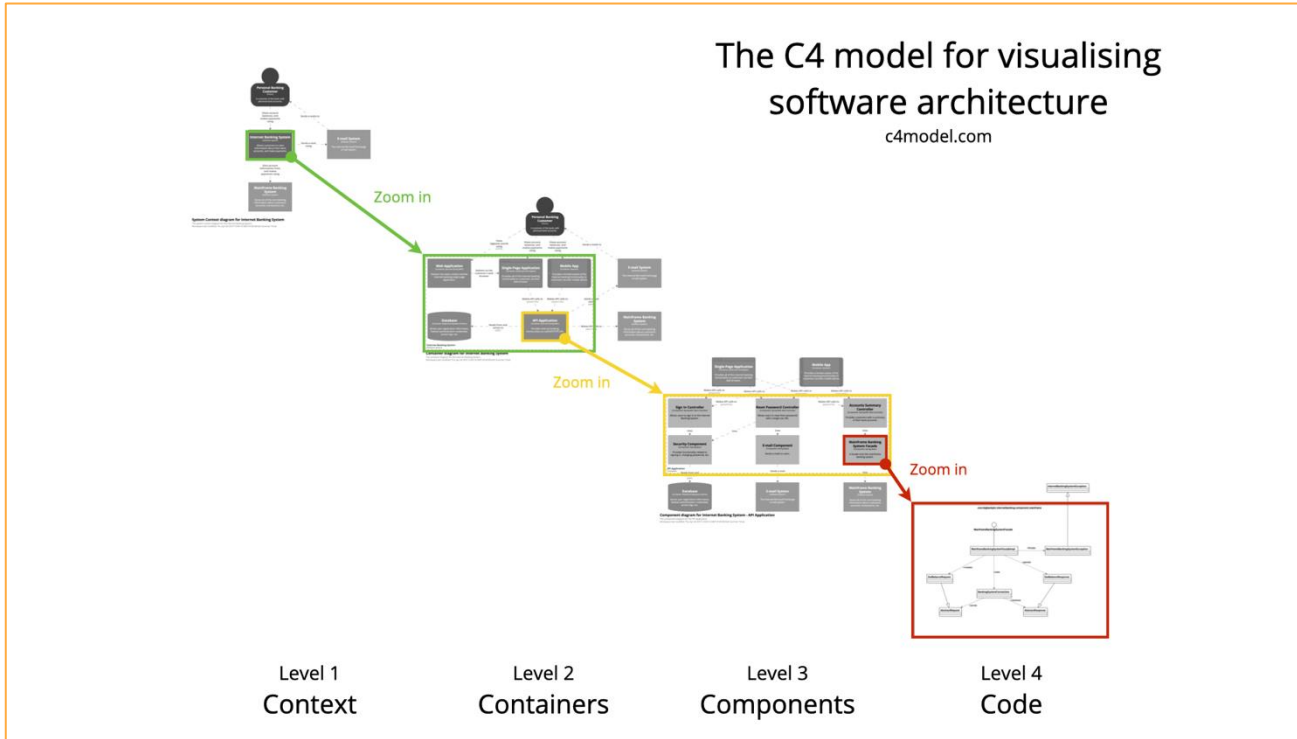
- **Contexto**: Diagrama de contexto o empresarial
- **Contenedores**: Estructura de alto nivel
- **Componentes**: zoom y descomponer
- **Código**: Diagramas de clases UML, diagramas ER, ...

## Modelo C4

### Guía de documentación:

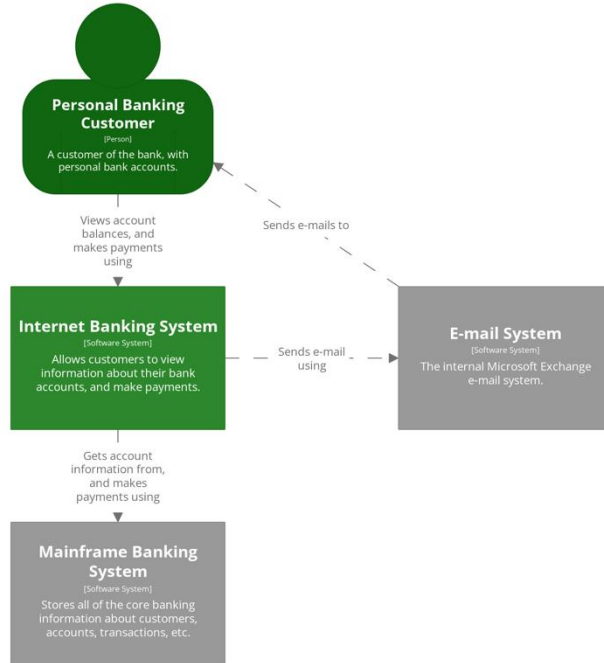
- ✓ Contexto
- ✓ Resumen funcional
- ✓ Atributos de calidad
- ✓ Restricciones
- ✓ Principios
- ✓ Código
- ✓ Datos
- ✓ Infraestructura de la arquitectura
- ✓ Despliegue
- ✓ Entorno de desarrollo
- ✓ Operaciones y soporte
- ✓ Log de decisiones

## Modelo C4



**Los diferentes niveles de zoom te permiten contar diferentes historias a diferentes audiencias.**

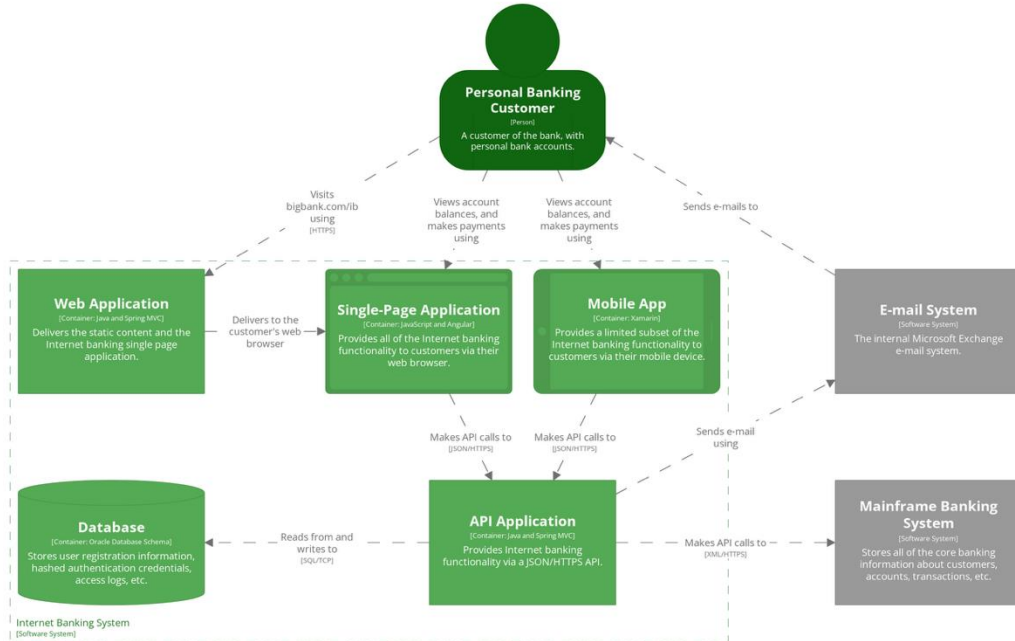
# Modelo C4 - Diagrama de contexto del sistema E nlace



**Un diagrama de contexto del sistema** es un buen punto de partida para diagramar y documentar un sistema de software, permitiéndole dar un paso atrás y ver el panorama general. Muestre su sistema como una caja en el centro, rodeado por sus usuarios y los otros sistemas con los que interactúa. Los detalles no son importantes aquí ya que esta es la vista ampliada que muestra una imagen grande del panorama del sistema. La atención debe centrarse en las personas (actores, roles, personas, etc.) y los sistemas de software en lugar de tecnologías, protocolos y otros detalles de bajo nivel.



## Modelo C4 - Diagrama de contenedor



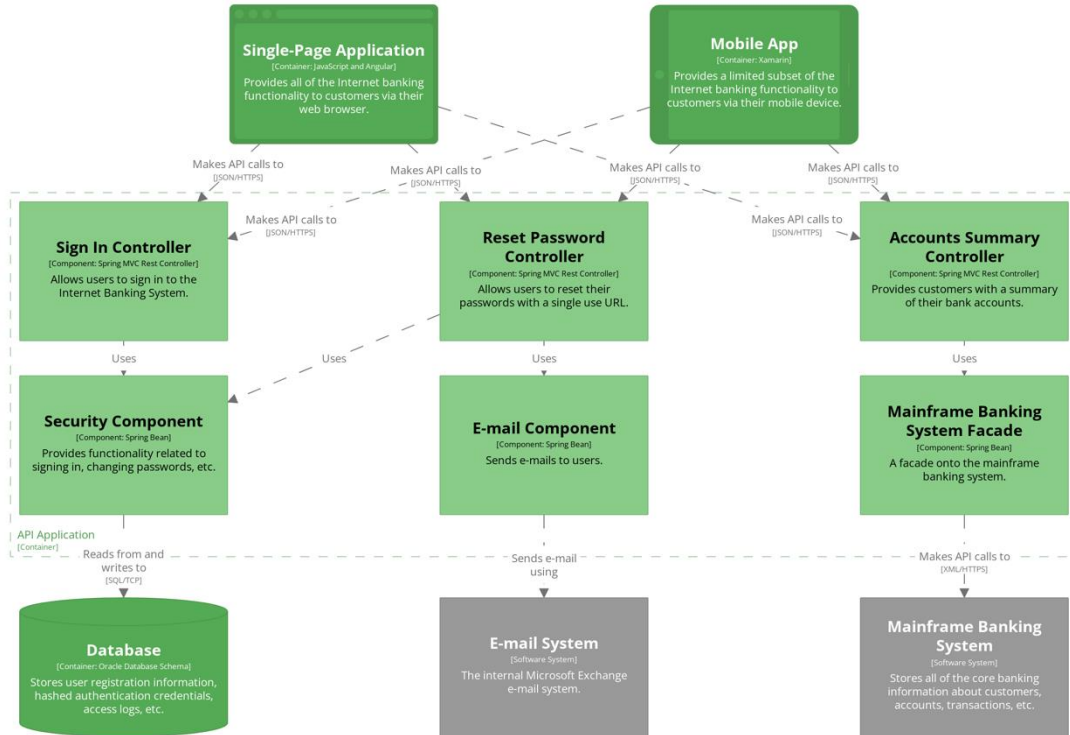
### [Container] Internet Banking System

The container diagram for the Internet Banking System - diagram created with Structurizr.  
Wednesday, March 22, 2023 at 8:16 AM Coordinated Universal Time

**Un diagrama de contenedor** muestra la forma de alto nivel de la arquitectura de software y cómo se distribuyen las responsabilidades en ella. También muestra las principales opciones tecnológicas y cómo los contenedores se comunican entre sí. Es un diagrama sencillo, centrado en la tecnología de alto nivel, que resulta útil tanto para los desarrolladores de software como para el personal de soporte/operaciones.

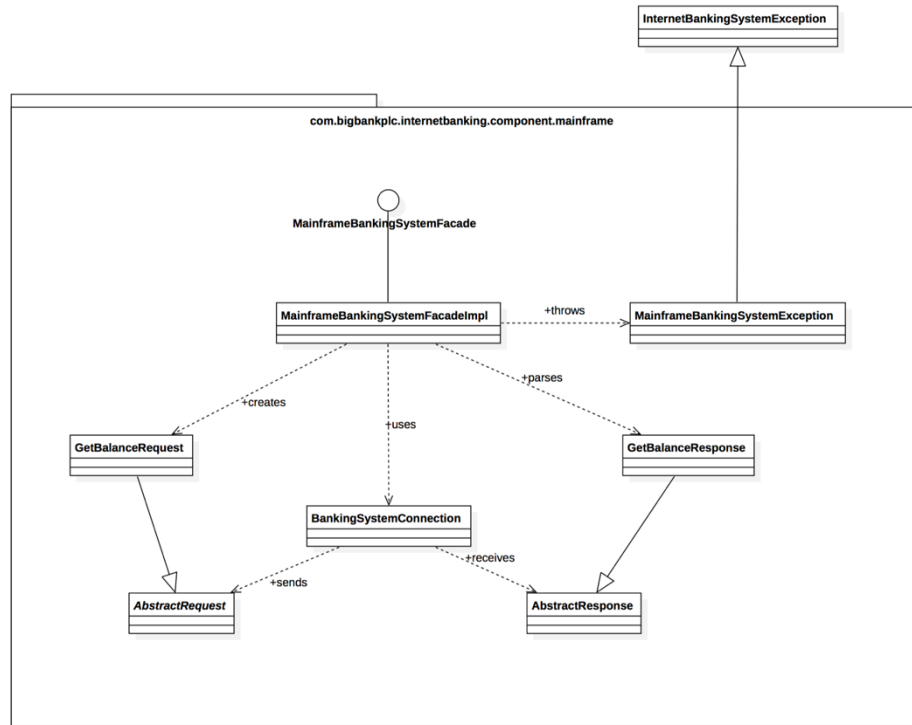


# Modelo C4 - Diagrama de componentes



El diagrama de componentes muestra cómo un contenedor se compone de una serie de "componentes", cuáles son cada uno de esos componentes, sus responsabilidades y los detalles de tecnología/implementación.

## Modelo C4 - Diagrama de código



Finalmente, puede ampliar cada componente para mostrar cómo se implementa como código; utilizando diagramas de clases UML, diagramas de entidades-relación o similares.

Este es un nivel de detalle opcional y, a menudo, está disponible bajo demanda desde herramientas como los IDE. Idealmente, este diagrama se generaría automáticamente utilizando herramientas (por ejemplo, una herramienta de modelado IDE o UML), y debería considerar mostrar sólo aquellos atributos y métodos que le permitan contar la historia que desea contar.

Este nivel de detalle no se recomienda para nada excepto para los componentes más importantes o complejos.

## Casos de Estudio



## Caso 2 :

En grupos de proyectos, desarrollen la documentación correspondiente al Sprint 1 de su proyecto utilizando el Modelo C4. Este modelo proporciona un enfoque estructurado para visualizar la arquitectura del software a través de diferentes niveles de detalle.

Comiencen creando un **Diagrama de Contexto**, que muestre cómo su sistema interactúa con actores externos y otros sistemas. Luego, elaboren un **Diagrama de Contenedores**, que detalle los principales contenedores del sistema, como aplicaciones, bases de datos y servicios, así como sus interacciones.

A continuación, desarrollen un **Diagrama de Componentes** para cada contenedor, donde se representen los componentes internos y sus relaciones. Finalmente, si es necesario, incluyan un **Diagrama de Código**, que ofrezca una vista más detallada de la implementación de un componente específico.

Asegúrense de documentar cada diagrama con descripciones claras y concisas que expliquen las decisiones de diseño y las interacciones entre los diferentes elementos. Esta documentación no solo servirá como guía para el desarrollo, sino que también facilitará la comunicación entre los miembros del equipo y otros interesados en el proyecto.



## **Caso 2 :**

Elaboren la documentación correspondiente al Sprint 1 de su proyecto utilizando el modelo 4+1 vistas. Este modelo le permitirá representar la arquitectura del software desde diferentes perspectivas, facilitando una comprensión integral del sistema.

# ¿Qué se logró?



- ✓ Aprender a elaborar documentación clara y organizada que represente la arquitectura del software, utilizando modelos como el 4+1 vistas para abordar diferentes aspectos del sistema.
- ✓ Adquirir la habilidad de identificar y describir las diferentes vistas arquitectónicas, entendiendo cómo cada una contribuye a una visión integral del sistema.
- ✓ Desarrollar competencias para comunicar efectivamente las decisiones de diseño y los componentes del sistema, facilitando la colaboración y el entendimiento entre los miembros del equipo y otros interesados en el proyecto.

# ¿Qué debemos hacer para alcanzar el objetivo del tema?



- ✓ Repasar la teoría, los ejemplos y casos resueltos.





## Referencias Bibliográficas

- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1995). Design Patterns: Elements of Reusable Object-Oriented Software Addison-Wesley.
- <https://github.com/RameshMF/gof-java-design-patterns>
- <https://refactoring.guru/design-patterns/abstract-factory/java/example>
- <https://refactoring.guru/es/design-patterns/abstract-factory/java/example>
- <http://www.cs.sjsu.edu/faculty/pearce/modules/patterns/creational/factory.htm>
- <https://www.oscarblancarteblog.com/2014/07/18/patron-de-diseno-factory/>

# GRACIAS